Лабораторна робота № 5

Тема. Практичне використання успадкування, поліморфізму, інкапсуляції та механізмів ітерації в класах.

Мета: навчитися використовувати успадкування, поліморфізм, інкапсуляцію та механізми ітерації в класах.

Короткі теоретичні відомості

Успадкування – це один з ключових принципів об’єктно-орієнтованого програмування, який дозволяє створити новий клас на основі вже існуючого класу. Головна мета використання успадкування:

1. повторне використання коду;
2. розширення функціоналу;
3. створення ієрархії.

Базовий синтаксис успадкування:

```
class Parent:
    pass

class Child(Parent):
    pass
```

Також мова підтримує множинне успадкування, завдяки чому похідний клас може успадковувати функціональність від кількох базових класів одночасно. Синтаксис:

```
class Base1:
    pass

class Base2:
    pass

class Child(Base1, Base2): # множинне успадкування
    pass
```

Найскладніший аспект множинного успадкування виникає, коли два батьківські класи мають спільного предка, що призводить до виникнення «проблеми ромба» (diamond problem). У такій ситуації постає питання, який саме метод слів викликати, якщо однойменний метод визначений у дочірньому класі, обох батьківських класах і їхньому спільному предку. Для

вирішення цієї проблеми використовується алгоритм MRO (Method Resolution Order), який визначає порядок пошуку методів і атрибутів у ланцюзі успадкування. Спочатку перевіряється дочірній клас, а потім батьківські класи у порядку їх переліку в оголошенні класу, тобто зліва направо. Перевірити порядок можна через атрибут *__mro__*. Приклад:

```
class A:
    def action(self):
        print("A")

class B(A):
    def action(self):
        print("B")
        super().action()

class C(A):
    def action(self):
        print("C")
        super().action()

class D(B, C):
    def action(self):
        print("D")
        super().action()

print(D.__mro__) # Вивід: (<class '__main__.D'>, <class
'__main__.B'>,
# <class '__main__.C'>, <class '__main__.A'>, <class 'object'>)

d = D()
d.action() # Вивід:
# D
# B
# C
# A
```

super() дозволяє звертатися до методів батьківського класу. У разі множинного успадкування *super()* звертається до наступного класу в ланцюжку MRO.

Поліморфізм – це принцип об’єктно-орієнтованого програмування, згідно з яким об’єкти різних класів здатні реагувати на однаковий інтерфейс (виклик методу або оператора) відповідно до власної внутрішньої реалізації, при цьому тип об’єкта визначає, яка саме реалізація буде виконана під час роботи програми.

Приклад поліморфізму через перевизначення методів:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def calculate_bonus(self):
        return self.salary * 0.1

class Developer(Employee):
    def calculate_bonus(self):
        return self.salary * 0.2

class Manager(Employee):
    def __init__(self, name, salary, team_size):
        super().__init__(name, salary)
        self.team_size = team_size

    def calculate_bonus(self):
        return super().calculate_bonus() + self.team_size * 100

def show_bonus(employee): # поліморфізм
    print(employee.name, "отримає бонус:",
employee.calculate_bonus(), " грн.")

emp = Employee("Петро", 20000)
dev = Developer("Оксана", 40000)
mgr = Manager("Іван", 45000, 5)

show_bonus(emp)
show_bonus(dev)
show_bonus(mgr)
```

Поліморфізм без успадкування (duck typing): «Якщо це виглядає як качка, плаває як качка і крякає як качка, то це качка». Приклад:

```
def describe(obj): # поліморфізм
    print(obj.info())

class Cat:
    def info(self):
        return "Кіт"

class Dog:
    def info(self):
        return "Пес"

describe(Cat())
describe(Dog())
```

Приклад власного поліморфізму операторів:

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # перевизначає __add__, щоб об'єкти можна було додати через +
    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector2D(4, 4)
v2 = Vector2D(1, 2)
v3 = v1 + v2
print(v3)
```

У прикладі використано f-рядок (*f""*) – механізм форматування рядків, який дозволяє вставляти значення змінних або результати виразів безпосередньо в текст за допомогою фігурних дужок *{}*.

Перевантажувати потрібно лише ті оператори, логіка яких є очевидною для класу.

Іноді потрібно примусити всіх нащадків реалізувати певний метод, для цього існує модуль *abc* (abstract base classes). Приклад поліморфізму через абстрактні класи:

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area():
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self): # обов'язкова реалізація
        return self.width * self.height

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

def print_area(shape: Shape):
    print(f"Площа фігури: {shape.area()}")

rect = Rectangle(10, 5)
circ = Circle(7)

print_area(rect)
print_area(circ)
```

Конструкція *shape: Shape* – це анотація типу (або підказка типу, type hint), складається з двох частин: *shape* – ім’я змінної або параметр функції; *:* – розділювач, який вказує на початок анотації; *Shape* – очікуваний тип (клас)

цієї змінної. Анотації були введені у Python 3.5 та вони виконують такі завдання: покращення читабельності коду; допомога IDE; статичний аналіз.

Абстракція – це принцип об'єктно-орієнтованого програмування, який полягає у виділенні головних, суттєвих характеристик об'єктів та приховуванні другорядних деталей. Вона дозволяє приховувати складну внутрішню реалізацію, залишаючи для зовнішнього використання лише простий і зрозумілий інтерфейс у вигляді набору публічних методів. Будь-який клас, який створено, вже є абстракцією реального об'єкта.

Інкапсуляція – принцип об'єктно-орієнтованого програмування, який полягає в об'єднанні даних (атрибутів) та методів, що працюють із цими даними, в єдиному класі, а також у прихованні внутрішнього стану об'єкта від прямого доступу зовні. Рівні доступу:

Рівень	Приклад	Пояснення
Публічний	*self.name*	Доступний з будь-якої частини програми.
Захищений	*self._salary*	Призначений для внутрішнього використання в класі або його підкласах.
Приватний	*self.__balance*	Використовує механізм name mangling (перейменування імен). Python автоматично змінює ім'я атрибута, наприклад *__balance* перетворюється *_ClassName__balance*. Ускладнює випадковий доступ ззовні класу та захищає його від прямого використання в дочірніх класах.

Приклад:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self._salary = salary
        self.__password = "!2@a3p"

    def show_name(self):
        return self.name
```

```
    def _calculate_bonus(self):
        return self._salary * 0.1

    def __check_password(self):
        return self.__password

emp = Employee("Петро", 3000)
print(emp.name)
print(emp.show_name())
```

Геттери та сеттери через декоратор *@property* дозволяють звертатися до методу так, ніби це звичайний атрибут, але всередині виконує логіку. Приклад:

```
class Person:
    def __init__(self, age):
        self.__age = age

    @property
    def age(self): # Геттер
        return self.__age

    @age.setter
    def age(self, new_age): # Сеттер
        if 0 < new_age < 150:
            self.__age = new_age
        else:
            print("Помилка: некоректний вік")

p = Person(25)
print(p.age)
p.age = 30
print(p.age)
```

Механізми ітерації в класах це засоби, які забезпечують можливість послідовного перебору елементів об’єктів класу за допомогою спеціальних методів *__iter__()* та *__next__()*. Приклад створення ітератора:

```
class Countdown:
    def __init__(self, start):
        self.current = start # поточне число
```

```
    def __iter__(self):
        return self

    def __next__(self):
        if self.current < 0:
            raise StopIteration # завершення відліку

        value = self.current
        self.current -= 1
        return value

countdown1 = Countdown(5)

print(next(countdown1))
print(next(countdown1))
print(next(countdown1))
print(next(countdown1))

for num in Countdown(3):
    print(num)
```

raise StopIteration – це команда котра створює та викликає виняток (помилку) *StopIteration*, щоб повідомити, що елементи в ітераторі закінчилися. Приклад створення ітерованого класу:

```
class Playlist:
    def __init__(self, songs):
        self.songs = songs

    def __iter__(self):
        return iter(self.songs)

my_music = Playlist(["Blank Space", "As it Was", "Believer"])

for song in my_music:
    print(song)
```

Якщо писати *__iter__()* як генератор (з *yield*), буде автоматично створено ітератор і не потрібно писати *__next__()* та запам'ятовувати стан. Приклад:

```
class Fibonacci:
    def __init__(self, count):
        self.count = count

    def __iter__(self):
        a, b = 0, 1
        for _ in range(self.count):
            yield a
            a, b = b, a + b

fib1 = Fibonacci(7)
for num in fib1:
    print(num)
```

Порядок виконання роботи

1. Уважно вивчити короткі теоретичні відомості та літературу.

2. Реалізувати власну ієрархію класів із використанням принципів успадкування, поліморфізму, інкапсуляції та механізмів ітерації в класах відповідно до власного варіанта.

3. Написати звіт з виконання лабораторної роботи.

Варіанти завдань

№ варіанта	Назва теми	Ієрархія класів	Ітерований клас
1	Бібліотека медіа	*MediaItem → Book, DVD, Magazine*	*Library*
2	Штат компанії	*Employee → Developer, Designer, Manager*	*Department*
3	Інвентар RPG гри	*Item → Weapon, Armor, Potion*	*Inventory*
4	Автопарк	*Vehicle → Car, Bike, Truck*	*Fleet*
5	Меню ресторану	*MenuItem → Appetizer, MainDish, Drink*	*Menu*
6	Навчальний курс	*CourseComponent → Lesson, Quiz, Assignment*	*Course*

7	Платіжна система	*Payment → CreditCard, BankTransfer, Crypto*	*TransactionHistory*

Контрольні питання

1. Що таке успадкування в ООП?
2. Що таке поліморфізм?
3. Як зробити атрибут приватним в Python?
4. Для чого використовується *super()* у дочірньому класі?
5. Який спеціальний метод робить клас ітерованим?